Wireless Network Simulation in NS3

S. Amirtharaj

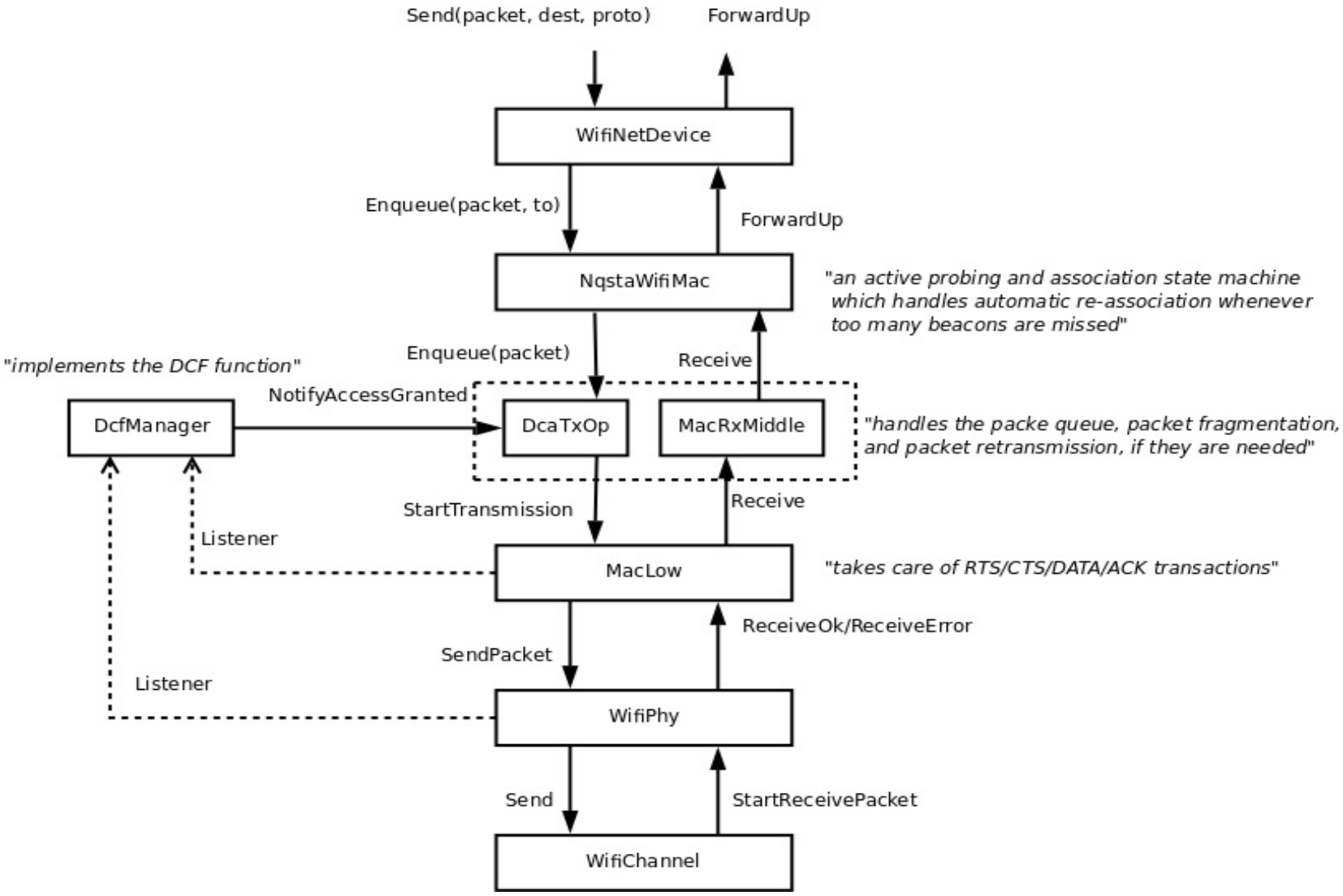
Network Topology

- Create a Simple Wireless network with two nodes Node0 and Node1. use IEEE 801.11 protocol to connect them.
- Develop a user defined application to send TCP packets from one node to another node , and install the application on node0.
- Install TCPSink application on node1.
- Start TCP application at time 1Second to send TCP packets to TCP sink.
- Trace the change in congestion window size
- Calculate various flow related performance metrics

Wifi Models Overview

- The set of 802.11 models provided in ns-3 attempts to provide an accurate MAC-level implementation of the 802.11 specification and to provide a not-so-slow PHY-level model of the 802.11a and 802.11b specifications.
- The current implementation provides roughly 4 levels of models:
- the PHY layer models
- the so-called MAC low models: they implement DCF and EDCAF
- the so-called MAC high models: they implement the MAC-level beacon generation, probing, and association state machines.
- a set of Rate control algorithms used by the MAC low models.

Wi-Fi Stack overview



Wifi Clasess

- NS3 Provides several Classes to construct wireless network topology using Wi-Fi
- WifiChannelHelper
  - The helper can be used to create a WifiChannel with a default PropagationLoss and PropagationDelay model
  - the default is a channel model with a propagation delay equal to a constant, the speed of light, and a propagation loss based on a log distance model with a reference loss of 46.6777 dB at reference distance of 1m.
  - Random Delay and Random loss models are also supported.

- Users will typically type code such as:

```
YansWifiChannelHelper wifiChannelHelper = YansWifiChannelHelper::Default ();
Ptr<WifiChannel> wifiChannel = wifiChannelHelper.Create ();
```

- YansWifiPhyHelper
  - We need to create Phy objects appropriate for the YansWifiChannel; here the YansWifiPhyHelper will do the work.
  - The user code is typically:

```
YansWifiPhyHelper wifiPhyHelper = YansWifiPhyHelper::Default ();
wifiPhyHelper.SetChannel (wifiChannel);
```

Work Out Example

```
WifiHelper wifi;
wifi.SetStandard (WIFI_PHY_STANDARD_80211b);
YansWifiChannelHelper wifiChannel;
wifiChannel.SetPropagationDelay("ns3::ConstantSpeedPropagationDelayModel");
wifiChannel.AddPropagationLoss("ns3::LogDistancePropagationLossModel");
YansWifiPhyHelper wifiPhy=YansWifiPhyHelper::Default();
NqosWifiMacHelper wifiMac = NqosWifiMacHelper::Default ();
wifiPhy.SetChannel(wifiChannel.Create());
wifi=WifiHelper::Default();
wifinet=wifi.Install(wifiPhy,wifiMac,wifiNodes);
```

WifiHelper

- Default(Void) – creates AdHoc MacLayer with ARF rate control algorithm , and configure 802.11a physical and MAC layer.
- EnableLogComponents(void) – enables all wifi related log components
- Install (WifiPhyHeler &, WifiMacHelper &, NodeContainer)
- SetStandard(Standard)- Set the wifi standard for the network

YansWifiChannelHelper

- Default()- Create a default channel with constant delay and log distance based loss
- Create(void)- Creates a channel with configured parameters.
- SetPropagationDelay(Name, Parameter1, value1….. parameter7, value7) – set the propagation delay model with predefined parameters.
- AddPropogationLoss(Name, Parameter1, value1….. parameter7, value7) – set propogation loss model with predefined parameters)

YansWifiPhyHelper

- Default() – Creates a default wifi physical channel
- void SetChannel (Ptr<YansWifiChannel > channel)- associates the channelparameters to this physical channel

Mobility Model

- **The MobilityModel interface:**
  - **– void SetPosition (Vector pos)**
  - **– Vector GetPosition ()**
- **• StaticMobilityModel**
  - **– Node is at a fixed location; does not move on its own**
- **• RandomWaypointMobilityModel**
  - **– (works inside a rectangular bounded area)**
  - **– Node pauses for a certain random time**
  - **– Node selects a random waypoint and speed**
  - **– Node starts walking towards the waypoint**
  - **– When waypoint is reached, goto first state**
- **• RandomDirectionMobilityModel**
  - **– works inside a rectangular bounded area)**
  - **– Node selects a random direction and speed**
  - **– Node walks in that direction until the edge**
  - **– Node pauses for random time**
  - **– Repeat**

```
MobilityHelper  mobility;
mobility.SetPositionAllocator(
                        “ns3::RandomDiscPositionAllocator",
                        "X", StringValue("100.0"),"Y",
                        StringValue("100.0"),
                        "Rho",StringValue("Uniform:0:30"));
mobility.SetMobilityModel("ns3::StaticMobilityModel");
 mobility.Install(wifiNodes);
```